

emb_visualisation

June 19, 2026

```
[ ]: import os

import torch
import pandas as pd
import numpy as np
import cv2
import sqlite3
import pickle

import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots

import ipywidgets as widgets
from IPython.display import display, clear_output, HTML

from sklearn.metrics import silhouette_score
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
from umap import UMAP

from pathlib import Path

ROOT = Path.cwd().parents[1]

IMAGE_DIR = ROOT / "images/emb_visuals"
CACHE_DIR = ROOT / "data/cache"

[2]: os.makedirs(IMAGE_DIR, exist_ok=True)
os.makedirs(CACHE_DIR, exist_ok=True)

[3]: embeds = torch.load(ROOT / "data/embeddings/base_embeds.pt", weights_only=False)
cls_tokens = embeds["cls_tokens"]
patches = embeds["patches"]

[4]: conn = sqlite3.connect(ROOT / "data/sql/metadata.db")

meta = pd.read_sql_query("SELECT * FROM meta", conn)
```

```

categories = pd.read_sql_query("SELECT DISTINCT category FROM meta",
    ↪conn)["category"].to_list()
types_per_cat = pd.read_sql_query("""
    SELECT category, COUNT(DISTINCT type) AS num_types
    FROM meta
    GROUP BY category
    """, conn)
max_types = types_per_cat["num_types"].max()

conn.close()

```

```

[5]: def load_or_compute(path, fn):
    if path.exists():
        print("Loading ", path)
        return np.load(path)

    print("Computing ", path)
    result = fn()
    np.save(path, result)
    return result

```

```

[6]: pca = PCA(n_components=2)
tsne = TSNE(n_components=2)
umap = UMAP(n_components=2, random_state=42)

train_mask = meta["split"] == "train"
train_meta = meta[train_mask].reset_index(drop=True)
test_meta = meta[~train_mask].reset_index(drop=True)

pca_2d = load_or_compute(CACHE_DIR / "cls_pca.npy", lambda: pca.
    ↪fit_transform(cls_tokens))
tsne_2d = load_or_compute(CACHE_DIR / "cls_tsne.npy", lambda: tsne.
    ↪fit_transform(cls_tokens))
umap_2d = load_or_compute(CACHE_DIR / "cls_umap.npy", lambda: umap.
    ↪fit_transform(cls_tokens))

```

```

Loading c:\Projects\UL-Summer-Bursary-2026\data\cache\cls_pca.npy
Loading c:\Projects\UL-Summer-Bursary-2026\data\cache\cls_tsne.npy
Loading c:\Projects\UL-Summer-Bursary-2026\data\cache\cls_umap.npy

```

```

[7]: pca_train = pca_2d[train_mask]
tsne_train = tsne_2d[train_mask]
umap_train = umap_2d[train_mask]

cls_fig = make_subplots(
    rows=1,
    cols=3,

```

```

        subplot_titles=("PCA", "t-SNE", "UMAP")
    )

    colors = px.colors.qualitative.Dark24

    category_colors = {cat: colors[i % len(colors)] for i, cat in
        ↪enumerate(categories)}
    plot_meta = [("PCA", pca_train, 1), ("t-SNE", tsne_train, 2), ("UMAP",
        ↪umap_train, 3)]

    for idx, (method_name, coords, col) in enumerate(plot_meta):
        for category in categories:
            mask = train_meta["category"] == category

            cls_fig.add_trace(
                go.Scattergl(
                    x=coords[mask, 0],
                    y=coords[mask, 1],
                    mode="markers",
                    name=category,
                    legendgroup=category,
                    showlegend=(method_name == "PCA"),
                    customdata=np.stack([
                        train_meta.loc[mask, "category"],
                        train_meta.loc[mask, "path"]
                    ], axis=1),
                    hovertemplate=(
                        "Category: %{customdata[0]}" +
                        "<br>Path: %{customdata[1]}" +
                        "<extra></extra>"
                    ),
                    marker=dict(
                        size=4,
                        color=category_colors[category]
                    )
                ),
                row=1,
                col=col
            )

    cls_fig.update_xaxes(title_text="PC1", row=1, col=1)
    cls_fig.update_yaxes(title_text="PC2", row=1, col=1)

    cls_fig.update_xaxes(title_text="t-SNE1", row=1, col=2)
    cls_fig.update_yaxes(title_text="t-SNE2", row=1, col=2)

    cls_fig.update_xaxes(title_text="UMAP1", row=1, col=3)

```

```

cls_fig.update_yaxes(title_text="UMAP2", row=1, col=3)

cls_fig.update_layout(
    title="CLS Embedding Projections",
    width=1600, height=600,
    legend=dict(
        orientation="h",
        yanchor="top",
        y=-0.1,
        xanchor="center",
        x=0.5
    )
)

cls_fig.show()
cls_fig.write_image(IMAGE_DIR / "cls_embedding_comparison.svg")

```

```

[8]: pca_test = pca_2d[~train_mask]
tsne_test = tsne_2d[~train_mask]
umap_test = umap_2d[~train_mask]

def_fig = make_subplots(
    rows=1,
    cols=3,
    subplot_titles=("PCA", "t-SNE", "UMAP")
)

plot_meta = [("PCA", pca_test, 1), ("t-SNE", tsne_test, 2), ("UMAP", umap_test, 3)]

for idx, (method_name, coords, col) in enumerate(plot_meta):
    for category in categories:
        mask = test_meta["category"] == category
        good_mask = mask & (test_meta["type"] == "good")
        bad_mask = mask & (test_meta["type"] != "good")

        def_fig.add_trace(
            go.Scattergl(
                x=coords[good_mask, 0],
                y=coords[good_mask, 1],
                mode="markers",
                marker=dict(size=4, color="blue"),
                name="Normal",
                showlegend=(method_name == "PCA" and category == categories[0]),
                legendgroup="Normal",
                customdata=np.stack([
                    test_meta.loc[good_mask, "category"],
                    test_meta.loc[good_mask, "type"],

```

```

        test_meta.loc[good_mask, "path"]
    ], axis=1),
    hovertemplate=(
        "Category: %{customdata[0]}" +
        "<br>Type: %{customdata[1]}" +
        "<br>Path: %{customdata[2]}" +
        "<extra></extra>"
    )
),
row=1,
col=col
)

def_fig.add_trace(
    go.Scattergl(
        x=coords[bad_mask, 0],
        y=coords[bad_mask, 1],
        mode="markers",
        marker=dict(size=4, color="red"),
        name="Defective",
        showlegend=(method_name == "PCA" and category == categories[0]),
        legendgroup="Defective",
        customdata=np.stack([
            test_meta.loc[bad_mask, "category"],
            test_meta.loc[bad_mask, "type"],
            test_meta.loc[bad_mask, "path"]
        ], axis=1),
        hovertemplate=(
            "Category: %{customdata[0]}" +
            "<br>Type: %{customdata[1]}" +
            "<br>Path: %{customdata[2]}" +
            "<extra></extra>"
        )
    ),
    row=1,
    col=col
)

```

```

def_fig.update_xaxes(title_text="PC1", row=1, col=1)
def_fig.update_yaxes(title_text="PC2", row=1, col=1)

```

```

def_fig.update_xaxes(title_text="t-SNE1", row=1, col=2)
def_fig.update_yaxes(title_text="t-SNE2", row=1, col=2)

```

```

def_fig.update_xaxes(title_text="UMAP1", row=1, col=3)
def_fig.update_yaxes(title_text="UMAP2", row=1, col=3)

```

```

def_fig.update_layout(
    title="CLS Embedding Projections(Normal vs Defective)",
    width=1600, height=600,
    legend=dict(
        orientation="h",
        yanchor="top",
        y=-0.1,
        xanchor="center",
        x=0.5
    )
)
def_fig.show()
def_fig.write_image(IMAGE_DIR / "def_normal.svg")

```

```

[9]: def computer_cat_metrics(cat_data): # take in that dictionary plus category
    ↪name and probbaly meta data can appened that too, can dfs be appende and
    ↪picklable?
    test_labels = cat_data["labels"]
    test_cls = cat_data["test"]
    good_mask = cat_data["mask"]

    sil = silhouette_score(test_cls, test_labels)

    train_centroid = cat_data["centroid"]
    test_good = test_cls[good_mask]
    test_def = test_cls[~good_mask]

    good_inter = np.mean(np.linalg.norm(test_good - train_centroid, axis=1))
    defect_inter = np.mean(np.linalg.norm(test_def - train_centroid, axis=1))

    sep_ratio = defect_inter / good_inter

    good_centroid = test_good.mean(axis=0)
    def_centroid = test_def.mean(axis=0)

    good_intra = np.mean(np.linalg.norm(test_good - good_centroid, axis=1))
    defect_intra = np.mean(np.linalg.norm(test_def - def_centroid, axis=1))

    return {
        "category": cat_data["name"],
        "silhouette": sil,
        "good_inter": good_inter,
        "defect_inter": defect_inter,
        "seperation": sep_ratio,
        "good_intra": good_intra,
        "defect_intra": defect_intra
    }

```

```

[10]: cls_results_path = CACHE_DIR / "cls_results.pkl"
cluster_results_path = CACHE_DIR / "cluster_results.csv"
global_sil_path = CACHE_DIR / "global_silhouette.csv"

if cluster_results_path.exists() and global_sil_path.exists():
    cluster_results = pd.read_csv(cluster_results_path)
    global_metrics = pd.read_csv(global_sil_path)

elif not cls_results_path.exists():
    print(f"Run {ROOT / 'notebooks/embedding_analysis/baseline.ipynb'} to
    ↪ create this file")

else:
    with open(cls_results_path, "rb") as f:
        cls_results = pickle.load(f)

    categories = meta.loc[train_mask, "category"].to_numpy()

    cls_sil = silhouette_score(cls_tokens[train_mask], categories)
    pca_sil = silhouette_score(pca_train, categories)
    tsne_sil = silhouette_score(tsne_train, categories)
    umap_sil = silhouette_score(umap_train, categories)

    global_metrics = pd.DataFrame([{
        "cls_silhouette": cls_sil,
        "pca_silhouette": pca_sil,
        "tsne_silhouette": tsne_sil,
        "umap_silhouette": umap_sil
    }])

    cat_data_list = []
    for cat in cls_results.keys():
        cat_meta = meta[
            (meta["category"] == cat) &
            (meta["split"] == "test")
        ].reset_index(drop=True)

        good_mask = cat_meta["type"].to_numpy() == "good"

        cat_data = {
            "name": cat,
            "mask": good_mask,
            "centroid": cls_results[cat]["centroid"],
            "test": cls_results[cat]["test"],
            "labels": cls_results[cat]["labels"]

```

```

    }

    cat_data_list.append(cat_data)

rows = [
    computer_cat_metrics(cat_data)
    for cat_data in cat_data_list
]

cluster_results = pd.DataFrame(rows)

global_metrics.to_csv(global_sil_path, index=False)
cluster_results.to_csv(cluster_results_path, index=False)

global_metrics

```

```

[10]:   cls_silhouette  pca_silhouette  tsne_silhouette  umap_silhouette
0      0.678885      0.736228      0.707808      0.749513

```

```

[11]: cluster_results

```

```

[11]:   category  silhouette  good_inter  defect_inter  seperation  good_intra  \
0    bottle    0.104942    7.315165    18.281244    2.499089    6.936717
1    cable     0.005631    17.824220    22.918423    1.285802    17.336464
2    capsule   -0.072153    8.865551    13.054923    1.472545    8.762616
3    carpet     0.042476    10.594268    19.784687    1.867490    8.606180
4    grid       0.075255    19.194643    22.477034    1.171006    18.273790
5    hazelnut   0.035357    17.768282    23.515533    1.323456    16.975283
6    leather    0.161544    11.415676    26.463194    2.318145    9.138710
7    metal_nut  0.008135    12.177728    15.678962    1.287511    11.461525
8    pill      -0.054110    11.466777    15.724665    1.371324    11.294194
9    screw      0.020685    16.529192    16.315851    0.987093    16.358180
10   tile       0.051925    13.828687    30.462530    2.202851    13.194847
11  toothbrush  0.010358    10.852009    19.439432    1.791321    10.421842
12  transistor  0.078401    13.137226    17.983105    1.368866    11.784468
13   wood       0.078673    21.327732    28.108618    1.317937    18.945900
14   zipper     0.054824    10.680493    17.839190    1.670259    10.037414

    defect_intra
0      13.918730
1      22.108633
2      12.570442
3      16.614160
4      19.301456
5      21.654016
6      17.966806
7      14.659186

```



```

8      15.098665
9      15.930414
10     25.503605
11     16.982058
12     16.518234
13     23.288950
14     14.769593

```

```

[12]: def column_stats(df, column):
        s = df[column]

        if "good" in column:
            best = df.loc[s.idxmin(), "category"]
            worst = df.loc[s.idxmax(), "category"]
        else:
            best = df.loc[s.idxmax(), "category"]
            worst = df.loc[s.idxmin(), "category"]

        return {
            "mean": s.mean(),
            "median": s.median(),
            "std": s.std(),
            "min": s.min(),
            "max": s.max(),
            "worst_category": worst,
            "best_category": best
        }

```

```

[13]: numeric_cols = cluster_results.select_dtypes(include="number").columns

summary = pd.DataFrame({
    col: column_stats(cluster_results, col)
    for col in numeric_cols
}).T

summary

```

```

[13]:
          mean      median      std      min      max  \
silhouette    0.04013    0.042476  0.058844  -0.072153   0.161544
good_inter    13.531843  12.177728  4.082906    7.315165  21.327732
defect_inter   20.536493  19.439432    5.0035  13.054923  30.46253
seperation     1.595646    1.371324  0.449951    0.987093   2.499089
good_intra     12.635209  11.461525  3.947926    6.936717   18.9459
defect_intra    17.79233   16.61416  3.790444   12.570442  25.503605

          worst_category best_category
silhouette          capsule          leather

```

good_inter	wood	bottle
defect_inter	capsule	tile
seperation	screw	bottle
good_intra	wood	bottle
defect_intra	capsule	tile

```
[14]: PLOT_DIR = IMAGE_DIR / "patch_plots"
HTML_CACHE = CACHE_DIR / "patch_plots"

patch_size = 14

def plot_patches():
    os.makedirs(PLOT_DIR, exist_ok=True)
    os.makedirs(HTML_CACHE, exist_ok=True)
    plot_figs = []

    for cat in categories:
        test_meta = meta[
            (meta["category"] == cat)
            & (meta["split"] == "test")
        ]

        test_samples = (
            test_meta
            .groupby("type", group_keys=False)
            .sample(n=1, random_state=42)
            .sort_values("type", key=lambda s: s.map(lambda x: (x != "good",
↪x))))
        )

        types = test_samples["type"].tolist()
        num_type = len(types)

        fig = make_subplots(
            rows=2,
            cols=num_type,
            subplot_titles=types,
            vertical_spacing=0.02
        )

        projection_traces = {"UMAP": [], "t-SNE": []}
        for col, (idx, record) in enumerate(test_samples.iterrows(), start=1):
            patch_labels = []

            path = str(ROOT / record["path"])

            if record["type"] == "good":
```

```

        patch_labels = [0] * 256

    else:
        mask_path = path.replace("\\test\\", "\\ground_truth\\").
        ↪replace(".png", "_mask.png")

        mask_img = cv2.imread(mask_path, cv2.IMREAD_GRAYSCALE)
        mask_img = cv2.resize(
            mask_img,
            (224, 224),
            interpolation=cv2.INTER_NEAREST
        )

        patch_coords = []
        for y in range(0, 224, patch_size):
            for x in range(0, 224, patch_size):
                patch = mask_img[y:y+14, x:x+14]
                patch_labels.append(int(np.any(patch > 0)))

                patch_coords.append((x // patch_size, y // patch_size))
        patch_labels = np.array(patch_labels, dtype=bool)
        patch_coords = np.array([
            (x, y)
            for y in range(16)
            for x in range(16)
        ])

        patches_umap = umap.fit_transform(patches[idx])
        patches_tsne = tsne.fit_transform(patches[idx])

        img = cv2.imread(path)
        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

        fig.add_trace(go.Image(z=img), row=1, col=col)

        fig.update_xaxes(visible=False, row=1, col=col)
        fig.update_yaxes(visible=False, row=1, col=col)

        projections = {"UMAP": patches_umap, "t-SNE": patches_tsne}
        for proj_name, dis in projections.items():
            fig.add_trace(
                go.Scattergl(
                    x=dis[~patch_labels, 0],
                    y=dis[~patch_labels, 1],
                    mode="markers",
                    marker=dict(size=4, color="blue"),
                    name="Normal",

```

```

        showlegend=(col == 1),
        legendgroup="Normal",
        customdata=patch_coords[~patch_labels],
        hovertemplate=
        (
            "Patch X: %{customdata[0]}"
            "<br>Patch Y: %{customdata[1]}"
            "<extra></extra>"
        ),
        visible=(proj_name=="UMAP")
    ),
    row=2,
    col=col
)
projection_traces[proj_name].append(len(fig.data) - 1)

fig.add_trace(
    go.Scattergl(
        x=dis[patch_labels, 0],
        y=dis[patch_labels, 1],
        mode="markers",
        marker=dict(size=4, color="orange"),
        name="Defects",
        showlegend=(col == 2),
        legendgroup="Defects",
        customdata=patch_coords[patch_labels],
        hovertemplate=
        (
            "Patch X: %{customdata[0]}"
            "<br>Patch Y: %{customdata[1]}"
            "<extra></extra>"
        ),
        visible=(proj_name=="UMAP")
    ),
    row=2,
    col=col
)
projection_traces[proj_name].append(len(fig.data) - 1)

n_traces = len(fig.data)

buttons = []

for proj_name in projection_traces:

    visible = [False] * n_traces

```

```

        # Always show images
        for i, trace in enumerate(fig.data):
            if trace.type == "image":
                visible[i] = True

        # Show traces for this projection
        for idx in projection_traces[proj_name]:
            visible[idx] = True

        buttons.append(
            dict(
                label=proj_name,
                method="update",
                args=[
                    {"visible": visible}
                ]
            )
        )

        fig.update_layout(
            width=220 * num_type,
            height=550,
            updatemenus=[
                dict(
                    buttons=buttons,
                    direction="down",
                    x=0.0,
                    y=1.15
                )
            ]
        )

        html_path = HTML_CACHE / f"{cat}_plot.html"

        fig.write_html(html_path)
        fig.write_image(PLOT_DIR / f"{cat}_plot.png")

        plot_figs.append(html_path)

    return plot_figs

```

```

[15]: if not (HTML_CACHE.exists()):
        plot_figs = plot_patches()

    else:
        html_files = list(HTML_CACHE.glob("*_plot.html"))

```

```

    if len(html_files) != len(categories):
        plot_figs = plot_patches()
    else:
        plot_figs = html_files

fig_dict = dict(zip(categories, plot_figs))

out = widgets.Output()

dropdown= widgets.Dropdown(
    options=list(fig_dict.keys()),
)

def show_plot(change=None):
    with out:
        clear_output(wait=True)

        path = fig_dict[dropdown.value]

        display(
            HTML(path.read_text(encoding="utf-8"))
        )

dropdown.observe(show_plot, names="value")

display(dropdown, out)
show_plot()

```

```

Dropdown(options=('bottle', 'cable', 'capsule', 'carpet', 'grid', 'hazelnut',
↳ 'leather', 'metal_nut', 'pill', ...

```

```

Output()

```